Complexity analysis of the RTIGER core optimizations

A per-step big-$\mathcal{O}$ accounting of the `optimize-julia-core` fork

Gemma Lab (`faustovrz/RTIGER`)

June 11, 2026

Abstract

This note derives, step by step, the asymptotic cost of the rigid-HMM (rHMM) Baum–Welch fit that RTIGER performs, and the big-$\mathcal{O}$ improvement contributed by each optimization in the `optimize-julia-core` fork. Equation numbers refer to the supplementary text of Campos-Martin *et al.* (2023).[1] Every optimization preserves the arithmetic of the model (Sections 2–4 of the supplement); only the cost of evaluating it changes. Two of the optimizations remove an asymptotic factor (the rigidity window r, and the marker count T in the emission line search), one removes a memory factor (the sample count N), and two are constant-factor (allocation) wins that leave the complexity class unchanged.

1 Notation and cost model

symbol	meaning
T	markers (positions) in one observation chain
C	number of chains (samples × chromosomes); $T_{\text{tot}} = \sum_c T_c$
N	number of samples (so C and T_{tot} grow $\propto N$)
$\lvert S\rvert = s$	number of hidden states (= 3: paternal / heterozygous / maternal)
r	rigidity; augmented rigid state space $S' = S \times \{1, \dots, r\}$, $\lvert S'\rvert = s\,r$
D	number of *distinct* emission pairs (k, n) in a chain
E	objective evaluations per state in one emission-M-step line search

Table 1: Cost parameters. For low-coverage genotyping data $D \ll T$: the read depth n is small, so the distinct (k, n) pairs number a few dozen regardless of T.

The emission is a Beta–Binomial (supplement §3.3), $\psi_s(k; n, a_s, b_s) = \binom{n}{k} \mathrm{B}(k{+}a_s, n{-}k{+}b_s)/\mathrm{B}(a_s, b_s)$, whose evaluation costs a handful of `lgamma` calls — the dominant scalar op of the fit. A full fit is

$$\text{cost} \;=\; (\#\text{EM iterations}) \times \sum_{c=1}^{C} \big(\text{E-step}_c + \text{M-step}_c\big),$$

so we analyse the per-iteration, per-chain cost and then note how it aggregates.

[1] Campos-Martin R, Schmickler S, Goel M, Schneeberger K, Tresch A. *Reliable genotyping of recombinant genomes using a robust hidden Markov model.* Plant Physiology 2023;192(2):821–836. `doi:10.1093/plphys/kiad191`; PMC10231367. https://doi.org/10.1093/plphys/kiad191

2 The auxiliary window product Ψ_k^t (`productpsi`)

The rigid recursions repeatedly need the windowed emission product (supplement Eq. 21)

$$\Psi_k^t \;=\; \prod_{u=\max(t-r+1,1)}^{t} \psi_k(o_u), \qquad 1 \le t \le T. \tag{21}$$

Computing each of the $T \cdot s$ values by re-multiplying its length-r window costs $\mathcal{O}(T\,s\,r)$. In log space $\log \Psi_k^t = \sum_u \log \psi_k(o_u)$ is a fixed-width sliding sum: maintain a running window, add the entering term and drop the leaving term in $\mathcal{O}(1)$ per step.

$$\boxed{\mathcal{O}(T\,s\,r) \;\longrightarrow\; \mathcal{O}(T\,s)} \qquad \text{(removes the rigidity factor } r\text{; commit } \texttt{d725651}\text{)}.$$

At the production rigidity $r = 250$ this is a $\sim r$-fold reduction of the Ψ term.

3 Emission log-pdf memoization (`getlogpsi`)

Each forward, backward and Viterbi step evaluates $\psi_s(o_t)$ at every position (Eqs. 22–27, 75). Done literally, that is $\mathcal{O}(T\,s)$ Beta–Binomial evaluations per iteration — each one several `lgamma` calls. But $\psi_s(o_t)$ depends on $o_t = (k_t, n_t)$ only, and there are only D distinct pairs. Memoizing $\log \psi_s$ over the distinct (k, n) replaces the expensive evaluations with cheap table lookups:

$$\boxed{\mathcal{O}(T\,s) \text{ Beta–Binomial evals} \;\longrightarrow\; \mathcal{O}(D\,s) \text{ evals} + \mathcal{O}(T\,s) \text{ lookups}}$$

i.e. the costly part shrinks by the factor T/D (commit `d725651`).

4 Emission M-step line search (the dominant cost)

The Beta–Binomial parameters are fit by a 1-D line search over τ_s that maximizes (supplement Eqs. 38, 43)

$$Q(\tau_s) \;\propto\; \sum_{c=1}^{C} \sum_{t=1}^{T_c} \gamma_s^{t,c}\, \log \psi_s\big(k_t^c;\, n_t^c,\, a_s = m_s\tau_s,\, b_s = (1-m_s)\tau_s\big). \tag{43}$$

Original. Each of the E line-search evaluations rebuilds a Beta–Binomial object and sums $\log \psi_s$ over all T_{tot} markers: $\mathcal{O}(E\,s\,T_{\text{tot}})$ Beta–Binomial constructions per iteration. The supplement notes the forward/backward/Viterbi kernels are the "time-critical" parts; in practice this repeated full-marker objective is what dominates — empirically $\sim 99.97\%$ of upstream runtime.

Optimized. Because the sum only depends on the data through the γ-weighted counts at each distinct pair, regroup it once:

$$Q(\tau_s) = \sum_{(k,n)\in\mathcal{D}} \underbrace{\Big(\sum_{t:\,(k_t,n_t)=(k,n)} \gamma_s^t\Big)}_{G_s(k,n)\ \text{(pre-summed once)}} \log \psi_s(k; n, a_s, b_s). \tag{1}$$

The weights $G_s(k, n)$ are accumulated in one $\mathcal{O}(s\,T_{\text{tot}})$ pass; each line-search evaluation then costs $\mathcal{O}(s\,D)$:

$$\boxed{\mathcal{O}(E\,s\,T_{\text{tot}}) \;\longrightarrow\; \mathcal{O}\big(s\,T_{\text{tot}} + E\,s\,D\big)}$$

Since $E\,D \ll T_{\text{tot}}$, the M-step collapses from "E passes over all markers, rebuilding objects" to "one binning pass" — a measured $\sim 1500\times$ at production scale (commit `16b8a65`). After this, the M-step is no longer the bottleneck.

5 Forward / backward and Viterbi (constant-factor)

The rigid forward (Eqs. 22–23), backward (Eqs. 26–27) and Viterbi (Eq. 75)

$$\varphi_k^t = \max_{j \in S} \begin{cases} \psi_k(o_t)\, a_{kk}\, \varphi_k^{t-1} & j = k \\ \Psi_k^t\, a_{jk}\, \varphi_j^{t-r} & j \neq k \end{cases} \tag{75}$$

each take a max/sum over s predecessors at every position, giving $\mathcal{O}(T\,s^2)$ per chain (the dwell-counter shifts for $k < r$ are $\mathcal{O}(1)$ copies once Ψ is available). The optimizations here — in-place scalar log-sum-exp (Eq. 80) and an allocation-free in-place arg-max — do *not* change the class:

$\mathcal{O}(T\,s^2) \;\longrightarrow\; \mathcal{O}(T\,s^2)$, constant-factor: $\sim 5\times$ (fwd/bwd, commit `44ed85b`), $\sim 12\times$ (Viterbi, `6cf85ca`),

with $\sim$90$\times$ fewer allocations in the forward/backward kernel. Once the emission M-step is fixed (§4), these $\mathcal{O}(T\,s^2)$ kernels are what the per-iteration cost reduces to — hence *linear in* T.

6 Memory: streaming the M-step (`eb79933`)

The M-step needs the data only through pooled sufficient statistics that are *sums* over chains (Eqs. 40–42): $\sum_c \zeta$, $\sum_c \gamma^1$, and the γ-weighted counts $G_s(k, n)$.

Original. `EM()` held, for *all* C chains at once, the per-chain arrays ζ $(T \times s^2)$, γ $(s \times T)$ and α, β, ψ $(sr \times T)$, pooling only at the end:

$$\text{peak} = \mathcal{O}\Big(\textstyle\sum_c T_c\,(s^2 + sr)\Big) = \mathcal{O}\big(N\,T\,(s^2 + sr)\big) \quad \text{(linear in } N\text{)}.$$

Streaming. Fold each chain's contribution into the running sums inside the E-step loop and discard its arrays immediately (the public `@Probabilities` slot, which nothing reads, is no longer filled):

$$\boxed{\mathcal{O}\big(N\,T\,(s^2 + sr)\big) \;\longrightarrow\; \mathcal{O}\big(T_{\max}\,(s^2 + sr)\big) + \mathcal{O}(T_{\text{tot}})}$$

peak scratch becomes one chain's worth, *independent of* N; the only N-linear residual is the input count matrices that must be held. Measured: $\sim$33 GB projected $\rightarrow\sim$3.6 GB at 1400×50k.

7 Summary

The per-step time and memory complexity is consolidated in the table below.

optimization	before (per iter)	after (per iter)	gain
Ψ window (`productpsi`)	$\mathcal{O}(Tsr)$	$\mathcal{O}(Ts)$	drop r
emission log-pdf (`getlogpsi`)	$\mathcal{O}(Ts)$ evals	$\mathcal{O}(Ds)$ evals	T/D
emission M-step (`16b8a65`)	$\mathcal{O}(EsT_{\text{tot}})$	$\mathcal{O}(sT_{\text{tot}} + EsD)$	$\sim$1500$\times$
forward/backward (`44ed85b`)	$\mathcal{O}(Ts^2)$	$\mathcal{O}(Ts^2)$	$\sim$5$\times$ (const.)
Viterbi (`6cf85ca`)	$\mathcal{O}(Ts^2)$	$\mathcal{O}(Ts^2)$	$\sim$12$\times$ (const.)
memory, streaming (`eb79933`)	$\mathcal{O}(NT(s^2{+}sr))$	$\mathcal{O}(T_{\max}(s^2{+}sr))$	drop N

Table 2: Per-EM-iteration time complexity (top) and peak-memory complexity (bottom). $s = 3$ is constant; the asymptotically meaningful factors are T, r, D, E and N.

8 Reconciliation with the measured scaling

A marker-scaling head-to-head on one shared 109,703-locus panel (decimated by odd index to five sizes, both cores run *to convergence*: `eps`= 0.01, $r = 2$, identical deterministic init) gives empirical per-iteration-mean exponents

$$\text{original} \approx \text{markers}^{2.20}, \qquad \text{optimized} \approx \text{markers}^{1.34}.$$

These match the analysis:

- **Optimized.** With the M-step reduced to a binning pass plus an $\mathcal{O}(EsD)$ search (D nearly flat), per-iteration cost is dominated by the $\mathcal{O}(Ts^2)$ forward/backward/Viterbi kernels — essentially *linear in* T.
- **Original.** Per-iteration cost is dominated by the emission line search $\mathcal{O}(EsT_{\text{tot}})$ with a Beta–Binomial rebuilt per marker, and E (the `Optim` evaluation count) is data-dependent and grows with scale. The result is an empirically super-linear, erratic per-iteration cost ($\sim$markers$^{2.2}$, with iteration-to-iteration spikes) rather than a clean $\mathcal{O}(T)$ — exactly what the grouping in §4 removes.

So per-iteration cost goes from empirically $\sim$quadratic and erratic to provably linear ($\mathcal{O}(Ts^2)$), and the full-fit speed-up *widens* with problem size: $\sim$**30**$\times$ at 6,857 markers/sample to $\sim$**405**$\times$ at 109,703 (3.4 h $\to$ 30 s, both in 19 EM iterations).

Equivalence. Crucially the speed-up changes nothing in the result. Run to convergence at all five sizes, the two cores take the *same* number of EM iterations ($5, 7, 11, 16, 19$ for opt and orig alike), return *identical* Viterbi paths (329,109/329,109 positions at the full panel; 0 mismatches at every size), and fitted parameters agree to $\leq 3 \times 10^{-6}$ — float summation-order, not an algorithmic difference. The optimized core is the same estimator, computed at $\mathcal{O}(T)$ instead of empirically $\mathcal{O}(T^2)$ in markers.

9 Parallelism: the E-step across chains (a constant, not an exponent)

The E-step is independent across chains (samples $\times$ chromosomes) — the M-step only pools their summed sufficient statistics — so it is embarrassingly parallel. With P worker threads the per-iteration wall time is

$$\mathcal{O}(Ts^2) \;\longrightarrow\; \mathcal{O}\Big(\tfrac{Ts^2}{P}\Big), \qquad P \leq \min(\text{Julia pool},\ \text{physical cores}),$$

i.e. the *constant* drops by P but the *exponent* is unchanged: on a log–log plot the optimized line shifts down by $\log P$ with the same slope. Parallelism cannot beat $\mathcal{O}(N)$ — dividing a fixed amount of work among a *bounded* number of workers is a constant-factor speed-up, not a complexity-class change. (A sub-linear exponent would need P to grow with the data; the chromosome is the natural, fixed parallel grain, so it does not.)

In practice the realized speed-up is well below P because the kernel is **memory-bandwidth bound** — it streams the large α, β, ζ arrays with little arithmetic per element, so a few cores saturate the shared bandwidth. Measured on a 4-performance-core Apple M4: $\sim$1.4$\times$ at $P = 4$ on both real *Arabidopsis* (15 chains) and maize (10 chains) data, stable across chain counts, with GC $\sim$2.5% (not the limiter) and $P = 8$ no better than $P = 4$. Determinism is preserved by a fixed-order reduce and a convergence guard band; `threads`=1 (default) is the bit-identical serial path and `threads`>1 is Viterbi-identical. Commits `664eb33`, `be437d1`.